CPO+ Unified Testbench Document (Unified Spec)

This document defines a single **CPO+ testbench** where three experiments—**Cycle (Babylonian)**, **Recursion (African fractals)**, and **CRT Recomposition (Chinese CRT)**—share the *same* core abstractions:

- **(a) Data generators** (periodic drift, multiscale textures, residue-factorizable constraints)
- **(b) Exact operators** $X_N, Z_N, U_{\mathrm{CRT}}$
- **(c) Recursive-net skeletons** (prime-branching hierarchical models)
- **(d) Multiplicity functionals** as **explicit tensor contractions** (partition functions / lift-counts)

The goal is to make every experiment "speak" the same language:

$$ \boxed{\text{CPO+} = (C, R, \otimes, \Pi, \mu)} $$

where:

- C: cyclic operator family (Weyl shifts/phases)
- R: recursion operator family (multiscale refinement)
- $\otimes$: tensorization rule (factor chart + leg structure)
- Π: projection/reconstruction (CRT chart change)
- μ: multiplicity functional (solution-volume / degeneracy)

---

0) Minimal dependencies and design constraints

Recommended stack: Python + NumPy + PyTorch (optional but convenient).

- Use **index maps** instead of materializing $N \times N$ matrices unless N is tiny.
- CRT unitary U_{CRT} is implemented as a **permutation of basis indices**.
- Multiplicity μ is expressed as **tensor contraction** via `einsum` / summations.

---

1) Common objects: Chart, Operators, Tensors

1.1 CRT Chart

Let

$$ N = \prod_{i=1}^k q_i, \quad q_i = p_i^{e_i}, \quad \gcd(q_i, q_j)=1\ (i\neq j). $$

A **CRT chart** is the bijection:

$$ \mathbb{Z}/N\mathbb{Z} \cong \prod_{i=1}^k \mathbb{Z}/q_i\mathbb{Z}, $$

implemented by two maps:

- `to_residues(x) -> (x1,...,xk)` where $x_i = x \bmod q_i$
- `from_residues((x1,...,xk)) -> x` (CRT reconstruction)

Reference CRT reconstruction:

$$ x \equiv \sum_{i=1}^k x_i\, N_i\, (N_i^{-1} \bmod q_i) \pmod N,\quad N_i = N/q_i. $$

On computational basis $|x\rangle$, with $\omega = e^{2\pi i/N}$:

$$ X_N|x\rangle = |x+1\ \mathrm{mod}\ N\rangle,\qquad Z_N|x\rangle = \omega^x |x\rangle. $$

We treat them primarily as **actions** on vectors (states) rather than dense matrices.

Define a basis reindexing:

$$ U_{\mathrm{CRT}}|x\rangle = |x\bmod q_1\rangle\otimes\cdots\otimes|x\bmod q_k\rangle. $$

This is a **permutation** of basis labels (up to a chosen lexicographic ordering of residue tuples).

---

2) Shared API (conceptual)

All experiments share these components:

1. `Chart(qs)`
2. `.N` total modulus
3. `.qs` list of coprime factors
4. `.to_residues(x)` / `.from_residues(xs)`
5. `.flatten_residues(xs)` / `.unflatten_residues(idx)` for tensor indexing
6. `WeylOps(N)`
7. `.apply_X(state, s=1)` applies X_N^s
8. `.apply_Z(state, t=1)` applies Z_N^t
9. `.apply_XZ(state, s, t)` applies $X_N^s Z_N^t$
10. `CRTUnitary(chart)`
11. `.apply_U(state)` maps $\mathbb{C}^N \to \bigotimes \mathbb{C}^{q_i}$ (as reshaped tensor)

12. `.apply_U_dag(tensor)` inverse mapping
13. `Multiplicity`
14. `mu_count(S_list, chart)` exact lift counts for hard constraints
15. `mu_free(phi_list, chart)` log-partition for soft constraints
16. `mu_cycle(phi_list, chart, k)` cyclic-weighted partition (for harmonic tests)

---

3) (a) Data generators

3.1 Periodic mixtures with phase drift (Cycle experiment)

We generate periodic time series that *intentionally* experience drift so the cyclic embedding is stress-tested.

Signal model:

$$ y(t) = \sum_{m=1}^M A_m \cos\big(2\pi f_m (t+\delta(t)) + \varphi_m\big) + \epsilon(t), $$

- $\delta(t)$ = slow phase drift (random walk or low-frequency trend)
- $\epsilon(t)$ = noise

Dataset outputs:

- `t`: time indices
- `y`: noisy drifted signal
- `meta`: true frequencies/phases for diagnostics

Generator knobs:

- `N_cycle` (e.g., 60, 420, learned factor product)
- drift strength, noise level, number of harmonics

3.2 Multiscale texture / fractal fields (Recursion experiment)

We need multiscale structure where hierarchical recursion should help.

Provide two generators:

(i) IFS fractal occupancy maps (fast, discrete)

- Choose affine maps $x \mapsto A_i x + b_i$
- Iterate points and rasterize to an image

(ii) Recursive midpoint displacement / multiscale random fields (fast, continuous-ish)

- Start with coarse grid
- Repeatedly refine with noise scaled by level

Dataset outputs:

- `X`: images/tensors (e.g., 1×H×W)
- `y`: labels (optional) or reconstruction target

3.3 Residue-factorizable constraint families (CRT + multiplicity experiment)

We generate local constraint potentials ϕ_i over residues $x \bmod q_i$, optionally with coupling terms.

Soft constraint family:

$$ \phi_i(r) = \exp\big(-\beta\, d_i(r)\big),\quad r\in \{0,\dots,q_i-1\} $$

where d_i is a distance-to-target function (e.g., Hamming distance to a subset, modular distance to a target residue).

Hard constraint family (indicator):

$$ \mathbb{1}_{S_i}(r) = \begin{cases}1 & r\in S_i\ 0 & \text{else}\end{cases} $$

Optional coupling (to test where CRT factorization breaks):

$$ \psi(x_1,\dots,x_k) = \exp\big(-\gamma\, g(x_1,\dots,x_k)\big) $$

where g can be a low-rank coupling to keep contractions feasible.

---

4) (b) Exact operators: reference implementations (index-native)

Below are reference skeletons. They're written to avoid allocating huge dense operators.

```
import math
import numpy as np

class Chart:
    def __init__(self, qs):
        self.qs = list(qs)
        self.N = int(np.prod(self.qs))
        # Precompute CRT reconstruction coefficients
        self.Ni = [self.N // q for q in self.qs]
```

```
        self.inv = [pow(self.Ni[i], -1, self.qs[i]) for i in
range(len(self.qs))]

    def to_residues(self, x: int):
        return tuple(int(x % q) for q in self.qs)

    def from_residues(self, xs):
        x = 0
        for i, (xi, q) in enumerate(zip(xs, self.qs)):
            x = (x + int(xi) * self.Ni[i] * self.inv[i]) % self.N
        return int(x)

    def flatten_residues(self, xs):
        # lexicographic flatten: (((x1)*q2 + x2)*q3 + x3)...
        idx = 0
        for xi, q in zip(xs, self.qs):
            idx = idx * q + int(xi)
        return int(idx)

    def unflatten_residues(self, idx: int):
        xs = []
        for q in reversed(self.qs):
            xs.append(int(idx % q))
            idx //= q
        return tuple(reversed(xs))


class WeylOps:
    def __init__(self, N: int):
        self.N = int(N)
        self.omega = complex(math.cos(2*math.pi/N), math.sin(2*math.pi/N))

    def apply_X(self, state: np.ndarray, s: int = 1):
        # (X^s state)[x] = state[x - s]
        s = int(s) % self.N
        return np.roll(state, shift=s)

    def apply_Z(self, state: np.ndarray, t: int = 1):
        # (Z^t state)[x] = omega^(t*x) state[x]
        t = int(t) % self.N
        x = np.arange(self.N)
        phase = np.power(self.omega, t * x)
        return phase * state

    def apply_XZ(self, state: np.ndarray, s: int, t: int):
        return self.apply_X(self.apply_Z(state, t), s)
```

```python
class CRTUnitary:
    def __init__(self, chart: Chart):
        self.chart = chart

    def apply_U(self, state: np.ndarray):
        # returns tensor shaped (q1,q2,...,qk)
        assert state.shape == (self.chart.N,)
        tensor = np.zeros(self.chart.qs, dtype=state.dtype)
        for x in range(self.chart.N):
            xs = self.chart.to_residues(x)
            tensor[xs] = state[x]
        return tensor

    def apply_U_dag(self, tensor: np.ndarray):
        # inverse map back to length-N vector
        state = np.zeros((self.chart.N,), dtype=tensor.dtype)
        it = np.nditer(tensor, flags=['multi_index'])
        for val in it:
            xs = it.multi_index
            x = self.chart.from_residues(xs)
            state[x] = val
        return state
```

Notes:

- `CRTUnitary.apply_U` is written for clarity, not speed. For performance, precompute `x -> multi_index` maps.
- For quantum-style experiments, you may want complex dtype.

---

5) (c) Model skeletons for prime-branch recursion nets

We define a *single* recursive core and plug in branching factor b (prime or otherwise). The model is intended for:

- multiscale texture classification
- multiscale reconstruction
- optionally as a learned factor graph aggregator

5.1 Conceptual operator form

A level-ℓ recursion combines b children into one parent:

$$ h^{(\ell+1)} = \mathcal{R}_b\big(h^{(\ell)}_1,\dots,h^{(\ell)}_b\big). $$

In a tensor-network view, $\mathcal{R}_b$ is an isometry/tensor W_ℓ.

5.2 PyTorch skeleton (tree recursion)

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class PrimeBranchCombine(nn.Module):
    def __init__(self, b: int, d_in: int, d_out: int):
        super().__init__()
        self.b = b
        self.lin = nn.Linear(b * d_in, d_out)

    def forward(self, children):
        # children: (batch, b, d_in)
        x = children.reshape(children.shape[0], self.b * children.shape[2])
        return F.gelu(self.lin(x))


class PrimeBranchTreeNet(nn.Module):
    def __init__(self, b: int, depth: int, d_leaf: int, d_hidden: int, d_out:
int):
        super().__init__()
        self.b = b
        self.depth = depth
        self.embed = nn.Linear(d_leaf, d_hidden)
        self.combines = nn.ModuleList([
            PrimeBranchCombine(b=b, d_in=d_hidden, d_out=d_hidden)
            for _ in range(depth)
        ])
        self.head = nn.Linear(d_hidden, d_out)

    def forward(self, leaf_feats):
        """
        leaf_feats: (batch, b^depth, d_leaf)
        returns: (batch, d_out)
        """
        h = F.gelu(self.embed(leaf_feats))  # (B, b^D, d_hidden)

        width = self.b ** self.depth
        for level in range(self.depth):
            # group leaves/partials into blocks of size b
            h = h.reshape(h.shape[0], width // self.b, self.b, h.shape[2])
            # apply combine to each block (vectorize via reshape)
            h2 = h.reshape(-1, self.b, h.shape[3])
            h2 = self.combines[level](h2)
            h = h2.reshape(h.shape[0], width // self.b, -1)
            width //= self.b
```

```
        root = h.squeeze(1)  # (B, d_hidden)
        return self.head(root)
```

Where the "culture prior" enters:

- Choose $b \in \{2, 3, 5, 7\}$ (prime) and compare to composite $b \in \{4, 6, 8\}$.
- Use data where multiscale structure matters.

5.3 Optional: isometry constraint (tensor-network flavored)

To make W_ℓ closer to an isometry, periodically orthonormalize the linear layer weight blocks (e.g., QR on reshaped matrices). This is optional but useful if you want "operator-native" behavior.

---

6) (d) Multiplicity functionals as explicit tensor contractions

Multiplicity μ quantifies "how many compatible global lifts exist," or, in soft form, "how much solution-volume exists." The testbench supports both.

6.1 Hard multiplicity: exact lift counts

Given sets $S_i \subseteq \mathbb{Z}/q_i\mathbb{Z}$, define:

$$ \mu_{\mathrm{count}} = |{x\in\mathbb{Z}/N\mathbb{Z}: x\bmod q_i\in S_i\ \forall i}|. $$

Because CRT is a bijection, if the only constraints are residue-local, then:

$$ \mu_{\mathrm{count}} = \prod_{i=1}^k |S_i|. $$

This is an important sanity check.

To express as an explicit contraction, represent indicators as tensors $I_i \in \{0, 1\}^{q_i}$. Then

$$ \mu_{\mathrm{count}} = \sum_{x_1=0}^{q_1-1}\cdots\sum_{x_k=0}^{q_k-1} \prod_{i=1}^k I_i[x_i]. $$

Tensor contraction form (einsum):

```
import numpy as np

def mu_count(indicators):
    # indicators: list of 1D arrays I_i of length q_i
    # explicit contraction: sum over all indices of product
    T = indicators[0]
    for I in indicators[1:]:
```

```
        T = np.tensordot(T, I, axes=0)  # outer product
    return int(T.sum())
```

6.2 Soft multiplicity: partition function and free multiplicity

Given local potentials $\phi_i : \{0, \dots, q_i - 1\} \to \mathbb{R}_{\geq 0}$, define

$$ Z = \sum_{x\in\mathbb{Z}/N\mathbb{Z}} \prod_{i=1}^k \phi_i(x\bmod q_i). $$

In CRT coordinates this becomes

$$ Z = \sum_{x_1,\dots,x_k} \prod_{i=1}^k \phi_i(x_i), $$

so it contracts as a pure outer-product sum:

$$ Z = \prod_{i=1}^k \left(\sum_{r=0}^{q_i-1}\phi_i(r)\right). $$

Define

$$ \mu_{\mathrm{free}} = \log Z. $$

```
import numpy as np

def mu_free(phis, eps=1e-12):
    Z = 1.0
    for phi in phis:
        Z *= float(np.sum(phi))
    return float(np.log(Z + eps))
```

6.3 Cyclic-weighted multiplicity (harmonic probe)

To connect to the cyclic/harmonic experiment, define a *Fourier-probed* partition:

$$ Z_k = \sum_{x\in\mathbb{Z}/N\mathbb{Z}} \omega^{k x} \prod_{i=1}^k \phi_i(x\bmod q_i),\quad \omega=e^{2\pi i/N}. $$

This generally does **not** factor unless additional structure exists, so the testbench computes:

- `Zk_direct` (baseline)
- `Zk_crt` (only if the phase term is expressible in separable CRT form for chosen charts; otherwise skip)

Direct computation is still feasible for small/moderate N (e.g., $N \leq 10^5$ in NumPy with vectorization).

---

7) Experiment modules (all share Chart + WeylOps + Multiplicity)

7.1 Experiment 1 — Cyclic codec under drift

Objective: show that mixed-radix cyclic embeddings improve robustness to drift at equal representational budget.

Common core used: `Chart`, `WeylOps`, `CRTUnitary` (optional), `mu_cycle` (optional harmonic probing).

Pipeline:

1. Generate drifted periodic signals `y(t)`.
2. Build embeddings:
3. Baseline: Fourier features (standard)
4. CPO-cycle: mixed-radix (e.g., 60 = 4×3×5) embedding and/or CRT-indexed harmonic tensor
5. Fit a simple predictor/autoencoder using identical capacity.
6. Measure reconstruction error vs drift strength.

Key evaluation artifacts:

- error vs drift curve
- bitrate/compression vs error

7.2 Experiment 2 — Prime-branch recursion nets

Objective: test whether prime-arity recursion gives measurable gains (not just aesthetics).

Common core used: recursion operators R implemented by `PrimeBranchTreeNet`, multiplicity optionally used as a regularizer (below).

Pipeline:

1. Generate multiscale textures.
2. Convert images to leaf features (patchify into b^D leaves).
3. Train `PrimeBranchTreeNet` for classification or reconstruction.
4. Ablate $b \in \{2, 3, 5, 7\}$ vs composites.

Optional multiplicity regularizer: encourage distributed solution-volume by penalizing overly peaked residue potentials derived from intermediate representations (turns μ into a training signal).

7.3 Experiment 3 — CRT "prime tensor" inference / reconstruction

Objective: demonstrate that CRT charting reduces inference cost when structure factorizes.

Common core used: `Chart`, `CRTUnitary` for basis reindexing, `mu_count`/`mu_free` as contraction engines.

Pipeline:

1. Choose N composite, e.g., $N = 15, 21, 35$ to start.
2. Generate local potentials ϕ_i over residues.
3. Compute multiplicity:
4. direct sum over $x \in \{0, \ldots, N-1\}$
5. CRT contraction over residue tensors
6. Verify equality (up to floating error), compare runtime.

---

8) Reference data generators (skeletons)

```python
import numpy as np

# 8.1 Periodic drifted signals

def gen_drifted_periodic(T=2048, M=5, noise=0.1, drift=1e-3, seed=0):
    rng = np.random.default_rng(seed)
    t = np.arange(T)
    A = rng.uniform(0.5, 1.5, size=M)
    f = rng.uniform(0.01, 0.2, size=M)
    phi = rng.uniform(0, 2*np.pi, size=M)

    # phase drift: random walk smoothed
    delta = rng.normal(0, drift, size=T).cumsum()

    y = np.zeros(T)
    for m in range(M):
        y += A[m] * np.cos(2*np.pi*f[m]*(t + delta) + phi[m])
    y += rng.normal(0, noise, size=T)

    meta = dict(A=A, f=f, phi=phi, delta=delta)
    return t, y, meta


# 8.2 Multiscale fields: recursive refinement

def gen_multiscale_field(H=64, W=64, levels=4, base_noise=1.0, seed=0):
    rng = np.random.default_rng(seed)
    field = rng.normal(0, base_noise, size=(H//(2**levels), W//(2**levels)))

    for lvl in range(levels):
        # upsample by 2 (nearest), then add finer noise
        field = field.repeat(2, axis=0).repeat(2, axis=1)
        scale = base_noise / (2**(lvl+1))
        field = field + rng.normal(0, scale, size=field.shape)
```

```
    # crop to exact HxW
    return field[:H, :W]


# 8.3 Residue potentials

def gen_residue_potentials(qs, beta=1.0, seed=0):
    rng = np.random.default_rng(seed)
    phis = []
    targets = []
    for q in qs:
        target = int(rng.integers(0, q))
        r = np.arange(q)
        # modular distance
        d = np.minimum((r-target) % q, (target-r) % q)
        phi = np.exp(-beta * d.astype(float))
        phis.append(phi)
        targets.append(target)
    return phis, targets
```

---

9) Sanity checks (must-pass tests)

1. **CRT invertibility**
2. For random x, `from_residues(to_residues(x)) == x`.
3. **Unitary is permutation** (for basis states)
4. For basis $|x\rangle$, `U_dag(U(|x>)) == |x>`.
5. **Multiplicity identity (local-only constraints)**
6. Hard: $\mu_{\text{count}} = \prod_i |S_i|$
7. Soft: $Z = \prod_i \sum_r \phi_i(r)$
8. **Weyl relations (optional check)**

$$ Z_N X_N = \omega X_N Z_N. $$

You can test numerically for small N.

---

10) Suggested run plan (small-to-large)

Stage A (tiny N, correctness):

- $N = 15 = 3 \cdot 5$, $N = 21 = 3 \cdot 7$
- validate CRT maps + multiplicity equality

Stage B (cycle codec):

- $N = 60$ as mixed-radix seed (4×3×5)
- drifted signal dataset

Stage C (recursion):

- prime branching $b \in \{2, 3, 5, 7\}$
- multiscale textures

Stage D (scale-up):

- larger CRT stacks (e.g., $N = 3 \cdot 5 \cdot 7 \cdot 11$)
- test chart speedups, memory, and failure modes with couplings

---

11) Extension hooks (optional but aligned with CPO+)

- **Cycle ↔ CRT bridge:** express harmonic probes Z_k in CRT coordinates when separable (chart-dependent).
- **Recursion ↔ multiplicity:** treat intermediate representations as residue-potentials; maximize μ under accuracy constraints to encourage "many consistent micro-realizations."
- **Operator learning:** learn combinations of Weyl operators $X^s Z^t$ as trainable layers (small N first).

Appendix: one-file "glue" outline

If you later want to turn this into a runnable script, the minimal structure is:

- `chart.py`: Chart + CRTUnitary
- `ops.py`: WeylOps
- `data.py`: generators
- `multiplicity.py`: mu_count / mu_free / mu_cycle
- `models.py`: PrimeBranchTreeNet
- `exp_cycle.py`, `exp_recursion.py`, `exp_crt.py`: experiment runners

But the spec above is designed so you can keep it all in one file initially.

References and further reading

Core number theory and CRT (modular decomposition / recomposition)

1. Kenneth Ireland & Michael Rosen, *A Classical Introduction to Modern Number Theory* (Graduate Texts in Mathematics 84), Springer, 2nd ed.
2. Wolfram MathWorld, "Chinese Remainder Theorem" (overview + pointers to standard texts).
3. H. L. Garner, "The Residue Number System," *IRE Transactions on Electronic Computers* EC-8(2):140–147 (1959). (Classic residue-number-system / CRT computation reference.)
4. S. R. Czapor & G. L. Labahn, *Algorithms for Computer Algebra*, Wiley (sections covering CRT + Garner-style reconstruction).

FFT / harmonic structure with CRT-style indexing

1. I. J. Good, "The interaction algorithm and practical Fourier analysis," *Journal of the Royal Statistical Society, Series B* (1958); addendum (1960). (Prime-factor / Good–Thomas lineage.)
2. J. W. Cooley & J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation* 19 (1965). (Cooley–Tukey FFT.)
3. Prime-factor FFT algorithm (Good–Thomas), standard description and references (Good 1958/1960; Thomas 1963; etc.).

Weyl–Heisenberg / generalized Pauli operators (Cycle operator basis)

1. Daniel Gottesman, "Fault-Tolerant Quantum Computation with Higher-Dimensional Systems," arXiv\:quant-ph/9802007 (1998). (Generalized Pauli / stabilizer theory for qudits.)
2. D. M. Appleby, "SIC-POVMs and the Extended Clifford Group," arXiv\:quant-ph/0412001 (2004). (Generalized Pauli group + Clifford normalizer structure.)

Tensor networks and hierarchical (tree) models

1. E. M. Stoudenmire & D. J. Schwab, "Supervised Learning with Tensor Networks," NeurIPS (2016), arXiv:1605.05775. (TN classifiers; optimization viewpoint.)
2. S. Cheng et al., "Tree tensor networks for generative modeling," *Physical Review B* 99, 155131 (2019). (TTN structure in ML.)
3. H. Chen et al., "Machine Learning with Tree Tensor Networks …," *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2024). (Modern TTN ML evidence.)
4. Tamara G. Kolda & Brett W. Bader, "Tensor Decompositions and Applications," *SIAM Review* 51(3):455–500 (2009). (General tensor tools and decompositions.)
5. I. V. Oseledets, "Tensor-Train Decomposition," *SIAM Journal on Scientific Computing* 33(5):2295–2317 (2011). (Practical low-rank tensor representation.)

Multiplicity as contraction / partition function (factor graphs)

1. F. R. Kschischang, B. J. Frey, H.-A. Loeliger, "Factor Graphs and the Sum-Product Algorithm," *IEEE Transactions on Information Theory* 47(2) (2001). (Partition functions and marginals as structured contractions.)

Fourier features / periodic embeddings (Cycle experiment baselines)

1. M. Tancik et al., “Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains,” NeurIPS (2020), arXiv:2006.10739.

Multiresolution / wavelets (Recursion baseline lens)

1. Stéphane Mallat, “A Theory for Multiresolution Signal Decomposition: The Wavelet Representation,” *IEEE TPAMI* 11(7) (1989).
2. Stéphane Mallat, “Multiresolution Approximations and Wavelet Orthonormal Bases of L2(R),” *Transactions of the AMS* 315(1) (1989).

Fractals / iterated function systems (Recursion data generators)

1. John E. Hutchinson, “Fractals and Self-Similarity,” *Indiana University Mathematics Journal* 30:713–747 (1981). (Foundational IFS formalism.)
2. Michael F. Barnsley, *Fractals Everywhere*, Academic Press (1988).

Cultural / historical mathematics context (bridge references)

1. Ron Eglash, *African Fractals: Modern Computing and Indigenous Design*, Rutgers University Press (1999).
2. Victor J. Katz (ed.), *The Mathematics of Egypt, Mesopotamia, China, India, and Islam: A Sourcebook*, Princeton University Press (2007).